

CyberAnalyst - Process Book

CyberAnalyst

COM-480 Data Visualization · EPFL 2026

Team members: Erik Hübner · Youcef Amar · André Cadet

Live site: <https://com-480-data-visualization.github.io/CyberAnalyst/>

Repository: <https://github.com/com-480-data-visualization/CyberAnalyst>

1. The Idea

There is an ever-going arms race between countries, companies, and individuals over data - be it bank records, military positions, or political influence. Cybersecurity incidents are documented and publicly available, yet their sheer volume and technical nature make them inaccessible to most people. An analyst can read individual incident reports, but understanding the macro-level shape of global cyber conflict - how it evolved over time, which nations bear the most pressure, and what it looks like on the ground in a country like Switzerland - requires synthesizing thousands of records at once.

Our goal was to build a data story that makes two decades of global cyber conflict readable to anyone: analysts catching up on trends, students learning about geopolitics, or simply curious citizens wondering whether Switzerland is as exposed as other nations.

We structured the narrative in three acts, moving from the global to the local:

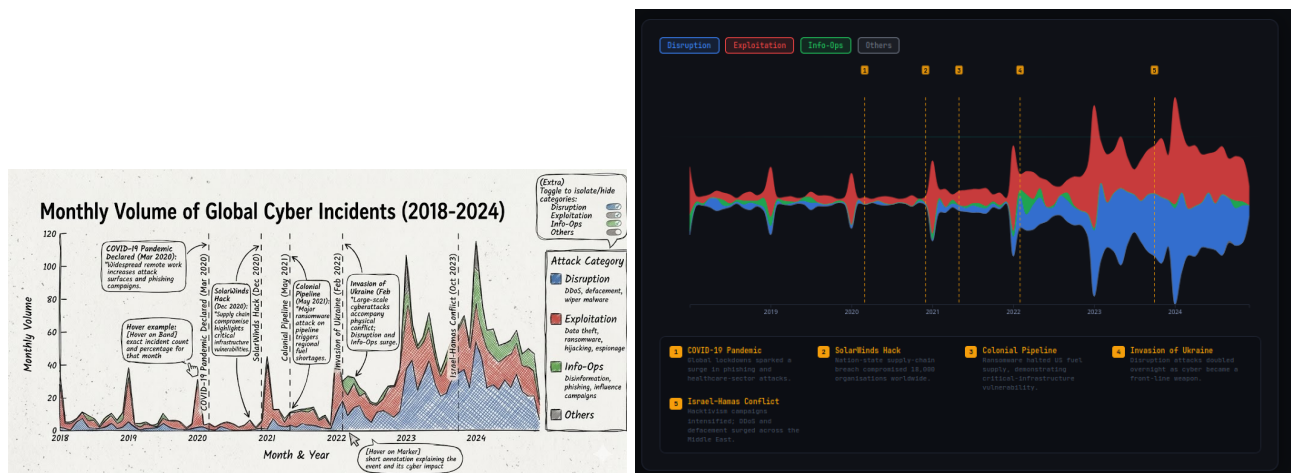
1. **The Global Pulse** - how attack volumes and categories evolved worldwide from 2018 to 2024, anchored to real geopolitical events
2. **Attack Intensity by Country** - which nations are the most targeted and who is behind the attacks
3. **Switzerland: A Domestic Deep Dive** - zooming into Swiss cantonal police data to reveal the structure of local cybercrime and the age groups most at risk

Datasets:

- **EuRepoC Global Cyber Incidents v1.3** - 6,000+ incidents from 2000–2024, with 60+ variables including attack type, actor, target country, and intensity score. Maintained by a consortium of European universities tracking state-sponsored and politically motivated cyber operations globally.
- **KTZH Kantonspolizei Zürich** - 14,087 cybercrime reports from 2017–2024, broken down by subcategory and victim age group. Published under an open data license by the Canton of Zurich.

2. From Sketches to Reality

Visualization 1 - Streamgraph: The Global Pulse



Left: original sketch (Milestone 1). Right: final D3 implementation.

What was planned: A streamgraph showing monthly incident volume stacked by attack category (Disruption, Exploitation, Info-Ops, Others), with vertical dashed markers annotating key geopolitical events. Users could hover on bands for exact counts and on event markers for context.

What changed and why:

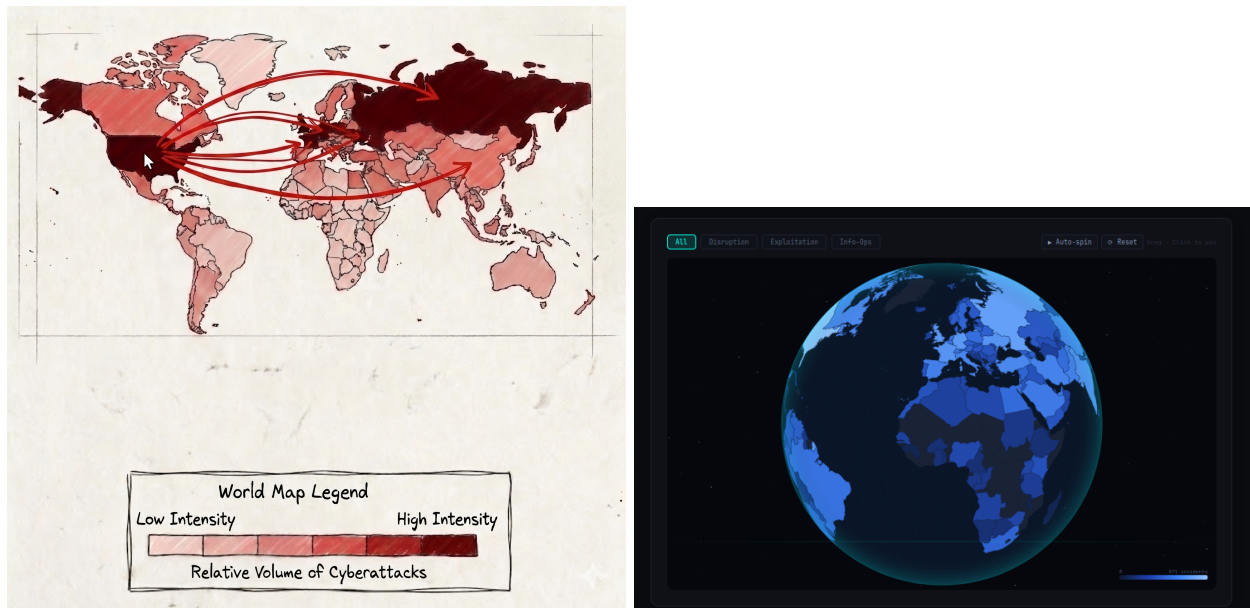
We switched from **Plotly.js** to **pure D3.js**. Plotly's abstraction layer made it impossible to control the streamgraph's offset algorithm - it defaulted to a stacked baseline which produced a hard-to-read shape. D3's `stackOffsetSilhouette` gives the organic, centered streamgraph that makes relative shifts between categories immediately visible.

The **event label design changed completely**. The sketch showed event names written directly on the chart. In practice, five labels on a narrow axis overlapped and became unreadable. We replaced them with numbered amber pins on the chart and a grid legend below - each entry links a number to a short description of the event and its cyber impact, keeping the chart clean while preserving contextual annotation.

Users can toggle individual attack categories on and off to isolate specific bands. This was not in the original sketch but emerged naturally from user testing - hiding Exploitation made the Disruption surge around the Ukraine invasion dramatically more visible.

Key insight the visualization reveals: Exploitation attacks dominated from 2018-2021, peaking during COVID when remote attack surfaces expanded. The Disruption band visibly doubles in February 2022 - the exact month of Russia's invasion of Ukraine - making the geopolitical correlation immediately readable without any annotation.

Visualization 2 - Globe: Attack Intensity by Country



Left: original flat choropleth sketch (Milestone 1). Right: final 3D rotatable globe.

What was planned: A flat choropleth world map colored by share of global attacks, with a dropdown filter by attack type, hover tooltips, and a click-to-pin detail panel showing a country's full attack breakdown.

What changed and why:

The most significant departure was replacing the **flat map** with a **3D rotatable globe**. The Mercator projection distorts area heavily - making Ukraine (one of the most attacked countries) appear tiny relative to Canada, which has far fewer incidents. It also felt static and unengaging for a global dataset spanning 160+ countries.

The orthographic globe addressed both issues. Countries appear at their true relative sizes, and the drag-to-rotate interaction naturally invites exploration. Users spin the globe to find their country, increasing time spent on the visualization and the likelihood of discovering less obvious patterns - for example, Southeast Asian countries targeted disproportionately for intellectual property theft.

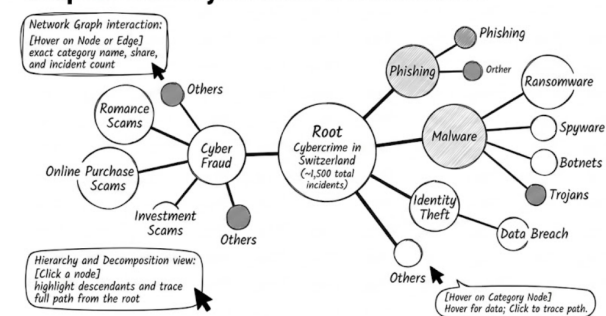
The dropdown filter was replaced by **toggle buttons** (All / Disruption / Exploitation / Info-Ops) for more immediate interaction. The click panel was upgraded to show the **top attributed threat actors** per country, a significantly more informative data point than a plain type breakdown.

A **logarithmic color scale** was necessary because the attack distribution is heavily right-skewed: the US has 371 incidents while the median country has fewer than 5. A linear scale would make all but a handful of countries appear identical.

Key insight the visualization reveals: The US and Ukraine absorb the largest share of global attacks. Switching to Exploitation mode shifts the dominant targets to Western nations, with China as the primary attributed actor. Switzerland, despite its neutrality, appears consistently targeted for economic espionage.

Visualization 3a - Network Graph: Swiss Cybercrime Structure

Deep Dive into Cybercrime in Switzerland



Left: original force-directed sketch (Milestone 1). Right: final fixed-depth layout.

What was planned: A node-link graph starting from a root node representing total cybercrime in Switzerland, branching into main categories and then subcategories, with node size proportional to incident count.

What changed and why:

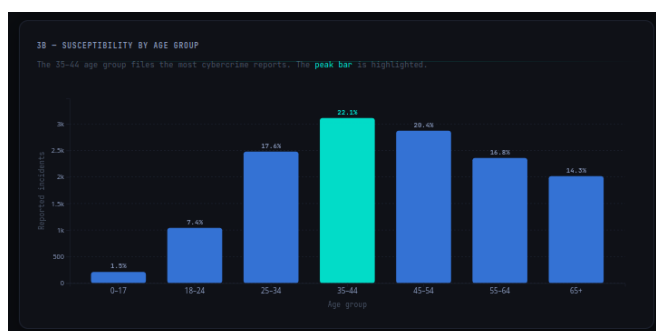
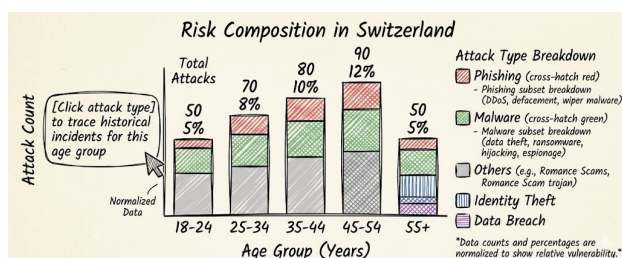
Force-directed layouts are non-deterministic: nodes settle in different positions each render, and the hierarchy is not immediately legible. For a 10-node tree-structured dataset this is particularly harmful - the depth relationship between root, category, and subcategory is the entire point of the visualization.

We switched to a **fixed depth-based layout**: depth 0 (root) at the top, depth 1 (main categories) in the middle row, depth 2 (subcategories) at the bottom. The hierarchical structure is instantly clear, and positions are stable across screen sizes.

The **click-to-highlight path** feature was not in the original sketch and was added during implementation. When a user clicks any node, all nodes outside its root-to-leaf path fade to near-invisible, and the connecting links glow. This lets users immediately answer questions like “what percentage of Swiss cybercrime is phishing?” with a single click - the answer is visible in the node label without any calculation.

Key insight the visualization reveals: Cyber Fraud dominates at 84% of all Swiss cybercrime reports. Within fraud, Phishing (4.3k), Investment Scams (3.2k), and Romance Scams (1.2k) are the three largest subcategories - all financial in nature, exploiting trust rather than technical vulnerabilities.

Visualization 3b - Bar Chart: Age Susceptibility



Left: original sketch (Milestone 1). Right: final implementation with peak highlight.

What was planned: A simple bar chart showing how different age ranges are affected by cybercrime in Switzerland, with the goal of showing which group is most exposed at a glance.

What changed and why:

The implementation stayed close to the sketch. The key design decision was **highlighting the peak bar** (35–44) in cyan (#00ffe7) while keeping all other bars in blue - the answer to “who is most at risk?” is visible in under a second without reading the axes. Percentage labels are placed on top of each bar so users can read exact figures

without referencing the axis ticks. Bars animate upward on page load, drawing the eye and communicating data as a progression rather than a static table.

Key insight the visualization reveals: The 35–44 cohort files 22.1% of all cybercrime reports - the highest of any age group. Under-25s are comparatively underrepresented (1.5% for 0–17, 7.4% for 18–24), likely because they have fewer financial assets to target and are more skeptical of investment and romance scams.

3. Technical Implementation

Stack: React 18 + Vite 5 (SPA), D3.js v7 (all visualizations), TopoJSON (world geometry), GitHub Actions (CI/CD), GitHub Pages (hosting).

Architecture

All four visualizations are independent React components, each managing their own D3 instance via `useRef` and `useEffect`. Data is loaded once in `App.jsx` and passed as props - this avoids duplicate fetch calls and keeps state management simple. Tooltips in every component are rendered via React's `createPortal` at the document root to avoid coordinate offset issues.

Challenge 1 - Timezone bug causing a flat streamgraph

The EuRepoC dataset stores timestamps as ISO strings ("2022-02-01T00:00:00.000000"). Our initial implementation parsed these with `new Date(iso).toISOString()` to use as map keys. In a UTC+2 timezone, midnight on February 1st becomes 10pm on January 31st after UTC conversion - shifting every date back by one day and causing all data lookups to return zero. The result was a completely flat chart with no visible data.

Fix: Use raw ISO strings directly as map keys (no parsing). Parse dates for D3's time scale as `new Date(s.slice(0, 10) + 'T12:00:00Z')` - noon UTC, immune to local timezone offset shifts.

Challenge 2 - Tooltips appearing far from the cursor

All four chart tooltips rendered inside positioned parent containers (`position: relative`). Tooltip coordinates were calculated in SVG space but the container had its own offset, placing the tooltip far from the cursor - sometimes in a completely different region of the page.

Fix: Use `createPortal(tooltip, document.body)` in every component. Tooltips render at the document root with `position: fixed` and coordinates taken from `event.clientX` / `event.clientY`, independent of any parent container.

Challenge 3 - Globe animation causing a black screen

The initial globe implementation called `svg.selectAll('*').remove()` followed by a full redraw on every animation frame (~60fps). Two catastrophic effects: React's event bindings were destroyed each frame (tooltips and clicks stopped working), and 120 background stars were re-randomized each frame (visible flickering).

Fix: Split the globe into two functions. `initSvg()` runs once on mount and handles the static background, stars, SVG groups, legend, and drag handler. `drawPaths()` runs each frame and updates only the rotating content (ocean sphere, graticule, country paths, atmosphere glow). Stars are pre-generated as a module-level constant so their positions never change.

Challenge 4 - Stale closures in D3 timer

The `d3.timer` callback captured `selectedType` state from the first render via JavaScript closure. When the user changed the attack-type filter, the globe continued rendering with the old color scheme because the closure saw the original value.

Fix: Maintain `useRef` values (`selectedTypeRef`, `intensityRef`, `dimsRef`) alongside each state variable. Each ref is kept in sync via `useEffect`. The timer callback reads from refs (always current) rather than from the stale closed-over state.

Challenge 5 - GitHub Pages deployment

Vite's dev server uses `index.html` referencing `/src/main.jsx`. GitHub Pages is a static host and cannot process `.jsx` files. Managing two versions of `index.html` (dev vs production) manually was error-prone and repeatedly caused the CI to deploy a broken page.

Fix: A GitHub Actions workflow builds the project on every push to `main` and deploys the compiled output directly to GitHub Pages. The dev `index.html` is always committed; the production bundle is built and deployed automatically. `git push` is the entire deployment process.

Challenge 6 - Streamgraph event listeners on transition

The D3 streamgraph attached `.on('mousemove', ...)` after `.transition()` in the method chain. In D3 v7, event listeners can only be attached to selections, not to transition objects. This caused a runtime error (`unknown type: mousemove`) that propagated up through React and crashed the entire component tree.

Fix: Move all `.on()` event handler calls before `.transition()` in the chain.

Challenge 7 - Kosovo ghost polygons accumulating on the globe

After rotating the globe, dozens of dark triangles appeared scattered across Russia's territory, each identifying as "Kosovo" on hover. Kosovo's TopoJSON feature has `id: undefined`. With `undefined` as the D3 data join key, D3 cannot match existing DOM elements on re-renders and appends a new `<path>` every animation frame (~60fps). After a few seconds of auto-spin, hundreds of orphaned paths had accumulated.

Fix: Filter features before the data join: `countries.features.filter(d => d.id !== undefined)`.

Challenge 8 - Hover highlight persisting after globe rotation

Hovering a country then rotating the globe left the cyan stroke permanently on that country. `mouseout` never fires when the globe rotates away — the browser only fires it when the pointer physically leaves the element. Stroke applied via `d3.select(this).attr('stroke', ...)` persisted indefinitely.

Fix: Track the hovered country in a `hoveredCountryRef`. Every `drawPaths()` frame re-applies stroke based on the ref, so a stale highlight cannot survive a rotation. The ref is cleared on drag start and during auto-spin. `drawPaths()` is called manually on `mouseover` and `mouseout` to keep the highlight instant when the globe is stationary.

4. Design Decisions

Dark cyber aesthetic: Background `#0a0c10`, accent `#00ffe7` (cyan), JetBrains Mono monospace font throughout. The palette reinforces the subject matter and gives the project a distinctive identity. The monospace font evokes a terminal or security dashboard without sacrificing readability at body text sizes. Every section uses a numbered badge with a pulse animation, reinforcing the "active monitoring" metaphor.

Consistent color system across all charts:

- Blue (`#3b82f6`) - Disruption attacks
- Red (`#ef4444`) - Exploitation attacks
- Green (`#22c55e`) - Info-Ops attacks
- Cyan (`#00ffe7`) - UI accent, peak highlights

The same four colors appear in the streamgraph bands, the globe's filter buttons, and the network graph node borders. Users build a mental model from the streamgraph that transfers directly to the globe's attack type toggles - they already know blue means disruption before they click the button.

Streamgraph over line chart: Simultaneously encodes total volume and composition shift in a single view. The Disruption band doubling in February 2022 is visible without any annotation. A multi-line chart would require mental arithmetic to reconstruct total volume from individual series.

Globe over flat map: Eliminates Mercator area distortion. The drag-to-rotate interaction invites exploration in a way a static map does not. Users who find their own country are more likely to click through to the attribution panel and engage with the underlying data.

Fixed layout over force-directed for network graph: Force-directed layouts are non-deterministic and suited for large, unstructured graphs. For a 10-node strict hierarchy, a fixed depth layout communicates the tree structure in one glance, with no ambiguity about which nodes are parents and which are leaves.

Logarithmic color scale for the globe: The attack distribution is heavily right-skewed. The US has 371 incidents; the median country has fewer than 5. A linear scale compresses all but three or four countries into an indistinguishable shade of near-zero.

Scroll-reveal and entrance animations: Sections fade up as they enter the viewport, giving the page a sense of progression as the user moves through the three-act narrative. Stat counters in the hero animate from zero on first scroll, giving the headline numbers a sense of scale and motion.

5. Challenges & Iterations

Plotly → D3 migration: We originally planned to use Plotly.js for the streamgraph and bar chart (documented in Milestone 2). During implementation it became clear that Plotly's abstractions prevented the interactions and visual style we needed - there was no way to control the stacking offset, the tooltip design, or the animation timing. The migration to pure D3 added approximately two weeks of development time but gave us complete control over every visual element.

Flat map → Globe rewrite: The flat choropleth was implemented, tested, and discarded. The Mercator distortion was not just an aesthetic issue - it was informationally wrong, misrepresenting the relative exposure of countries that are central to the dataset. Rewriting as a globe required learning D3's orthographic projection, implementing drag-based rotation with pitch clamping (to prevent the globe from flipping upside down), and separating static from dynamic SVG layers for smooth 60fps animation.

Animation debugging one by one: After adding all animations at once, the site went black. We reverted every animation and re-added them individually to isolate the cause. The root cause was the `IntersectionObserver` scroll-reveal: sections started at `opacity: 0` and the observer was configured with `threshold: 0.12`, meaning a section had to be 12% visible before becoming revealed. During initial load, the page wasn't tall enough (data hadn't loaded yet, so the charts were not rendered), so the observer never fired for any section and the page remained blank. Fixed by setting `threshold: 0` - sections reveal as soon as any pixel enters the viewport.

React StrictMode: In development, React StrictMode double-invokes all `useEffect` calls to detect side effects. This caused `initSvg()` to run twice: the second invocation cleared the SVG and ran a new initialization cycle immediately after the first `drawPaths()` had populated it, resulting in a blank globe and broken event handlers. We removed StrictMode and added an explicit error boundary that renders stack traces during development instead.

Data volume and fetch strategy: The EuRepoC dataset in raw form is several megabytes of JSON with 60+ columns per incident, most of which are irrelevant to our visualizations. We preprocessed it into two lean files: `graph1.json` (monthly aggregates by attack type for the streamgraph) and `country-intensity.json` (per-country totals). This reduced the total data transferred on page load from ~8MB to under 200KB.

6. Peer Assessment

All three team members contributed equally to the project (approximately one third each). Work was divided by visualization ownership, with shared responsibility for data processing and narrative design.

Team Member	Contributions
Erik Hübner	Website architecture (React + Vite), Streamgraph (D3 implementation, event markers, category toggles), deployment pipeline (GitHub Actions, GitHub Pages), scroll-reveal and entrance animations, UI polish, process book
Youssef Amar	Globe and choropleth map (D3 orthographic projection, drag rotation, logarithmic color scale, country attribution panel), EuRepoC data processing pipeline, attack-type filtering logic, country-sources data

Team Member	Contributions
André Cadet	Network graph (fixed-depth layout, click-to-highlight path), age susceptibility bar chart, KTZH Swiss dataset processing, narrative copy for all three sections, insight cards, data analysis and key findings

Datasets:

- EuRepoC Cyber Incident Dataset v1.3 - <https://eurepoc.eu>
- KTZH Kantonspolizei Zürich Cybercrime Statistics - <https://data.stadt-zuerich.ch>

Technologies:

- D3.js v7 - <https://d3js.org>
- React 18 - <https://react.dev>
- Vite 5 - <https://vitejs.dev>
- TopoJSON - <https://github.com/topojson/topojson>
- GitHub Actions - <https://docs.github.com/actions>
- GitHub Pages - <https://pages.github.com>